

Deterministic

Each (state, label) pair has **exactly one** target state.

- Given a state s and a label a , at most one transition $s \xrightarrow{a} s'$ exists.
- Behavior is fully predictable: the same input sequence always produces the same execution.
- Verification is simpler — box and diamond modalities coincide.

```
// Deterministic: one target per label
transition Locked -> Unlocked
  on label unlock;
transition Locked -> Strike1
  on label bad_code;
```

Non-Deterministic

The same (state, label) pair can have **multiple** target states.

- $s \xrightarrow{a} s_1$ and $s \xrightarrow{a} s_2$ with $s_1 \neq s_2$.
- Models uncertainty, environment choices, or abstraction.
- Execution is not uniquely determined by input alone.

```
// Non-deterministic: two targets for 'req'
transition Idle -> Granted
  on label req;
transition Idle -> Denied
  on label req;
```

Box vs Diamond

Box ($[\] \ \phi$) — universal: *all* successors satisfy ϕ .
Diamond ($\langle \rangle \ \phi$) — existential: *some* successor satisfies ϕ .

Syntax	Meaning
$[\text{labels} = \{a\}] \ \phi$	All a -transitions lead to ϕ
$\langle \text{labels} = \{a\} \rangle \ \phi$	Some a -transition leads to ϕ

Deterministic systems: box = diamond (only one successor), so both modalities are equivalent.
Non-deterministic systems: box is strictly stronger than diamond.

```
// Box: ALL req-transitions reach Safe
[ labels = {req} ] Safe

// Diamond: SOME req-transition reaches Safe
< labels = {req} > Safe
```

If there are two *req*-successors and only one is **Safe**, diamond holds but box fails.

Impact on Verification

Non-determinism affects formal verification in several ways:

- Box modality may fail** — a single “bad” successor is enough to violate $[\] \ \phi$.
- Diamond still holds** — only one “good” successor is needed for $\langle \rangle \ \phi$.
- Larger effective state space** — branching increases the number of paths to explore.
- Controller synthesis** — the controller must handle *all* non-deterministic branches the environment may take.

When to Use Non-Determinism

- Environment uncertainty** — the system cannot predict which branch the environment will choose.
- Protocol variations** — multiple valid responses to the same stimulus.
- Abstraction** — collapsing implementation details introduces non-determinism.
- Under-specification** — leaving design choices open.

Pitfall

Confusing **box** (all paths) with **diamond** (some path) when the system is non-deterministic. A property verified with diamond does *not* guarantee it holds on every execution — use box for universal guarantees.